



Datengetriebene Optimierung von PID-Reglern im LTI-Bereich mittels Machine Learning

Author: Niklas Keßel

Betreut durch: Prof. Dr. Ing. André Edelmann & Dipl.-Ing. Sameh Ben Nablia

Gliederung

1. Motivation & Problemstellung
2. Methodik
3. Ergebnisse
4. Diskussion
5. Fazit & Ausblick
6. Fragen & Diskussion

Motivation & Problemstellung

Bedeutung

PID-Regler sind in >90% aller Regelkreise im Einsatz.

Herausforderung

Manuelle Einstellung ist zeitaufwendig und erfordert Expertise.

Klassische Methoden

Guter Startpunkt, aber oft nicht optimal.
(ZN, CHR)

Statische Limitationen

Nicht anpassungsfähig an Prozessänderungen.

Flexibilität

Begrenzte Flexibilität bei Zielkonflikten.

Abbildung 1: Motivation (Eigendarstellung)

Zielsetzung & Forschungsfrage

- Forschungsfrage:
- Kann ein Machine Learning Ansatz die PID-Parametrierung im Vergleich zu klassischen Methoden verbessern?
- Ablauf der Arbeit:

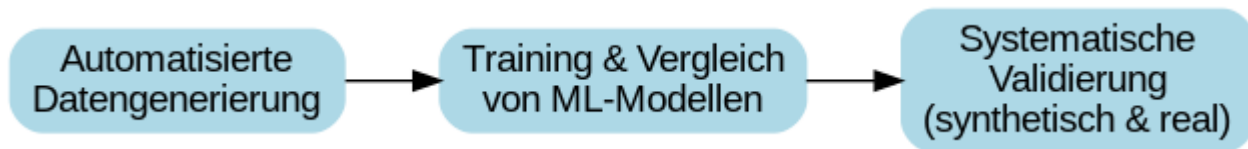


Abbildung 2: Vorgehensweise (Eigendarstellung)

Methodik 1: Die Machine-Learning-Pipeline

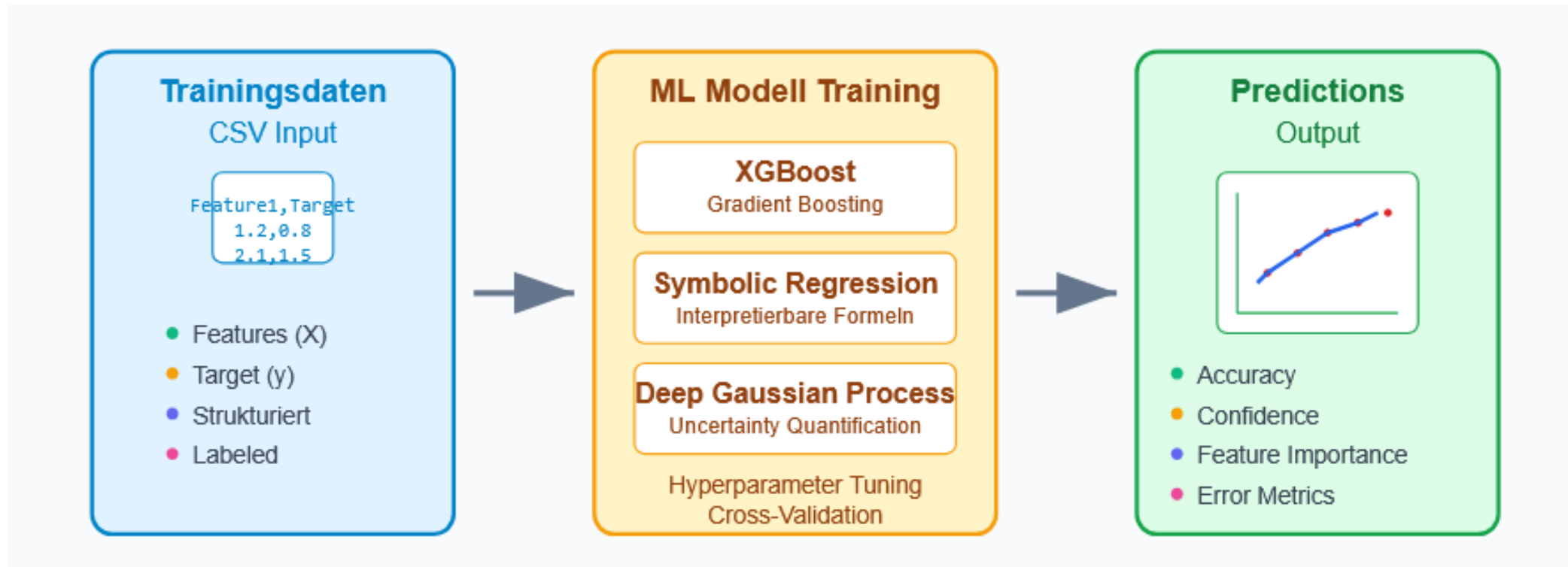


Abbildung 3: ML-Pipeline1 (Eigendarstellung)

Methodik 1: Die Machine- Learning- Pipeline

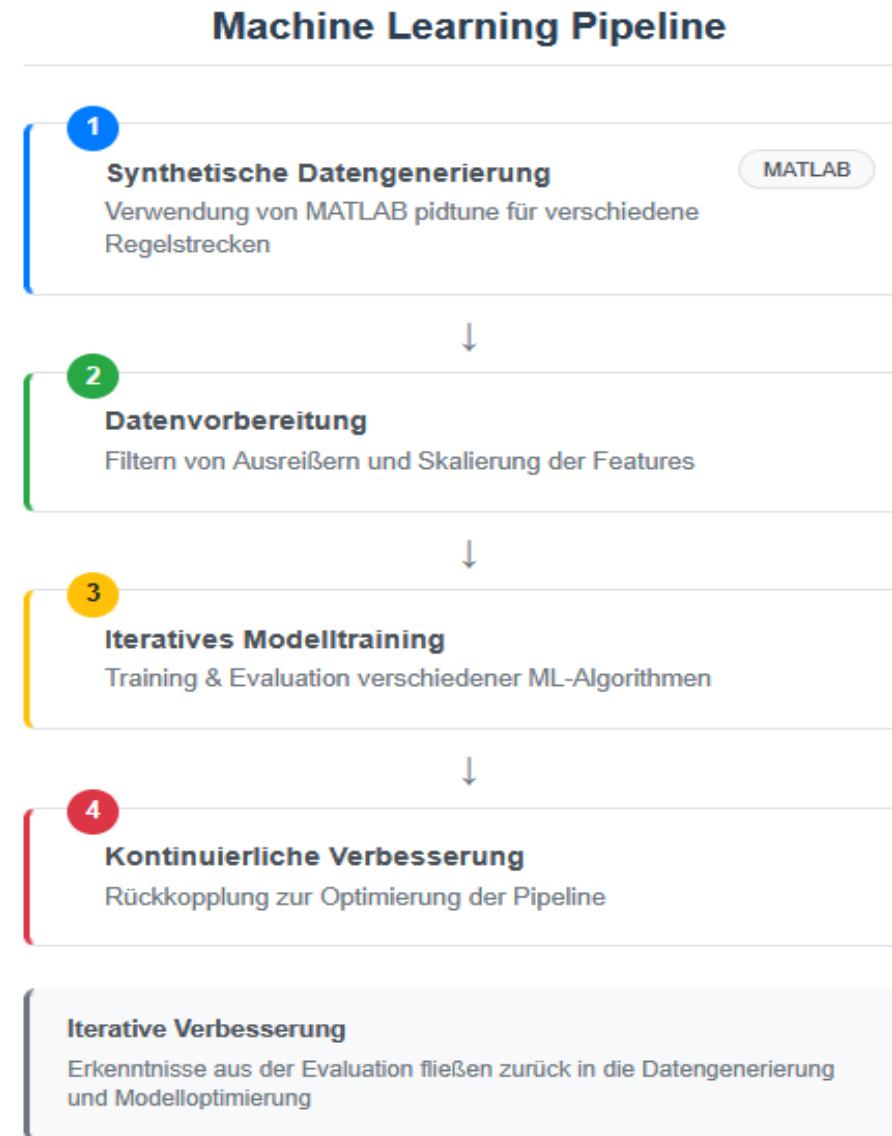


Abbildung 4: ML-Pipeline2 (Eigendarstellung)

Methodik 1: Die Machine- Learning- Pipeline

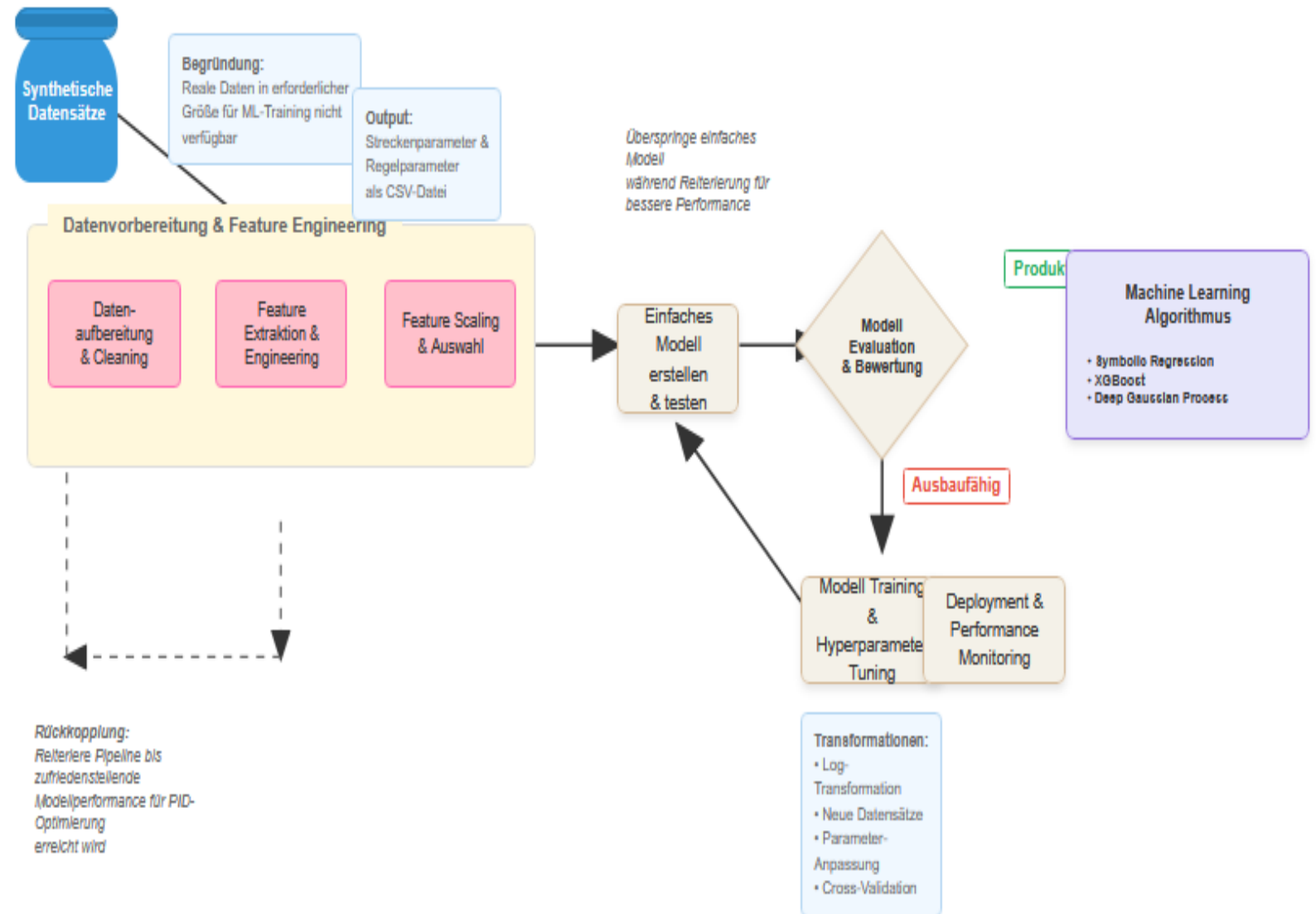


Abbildung 5: ML-Pipeline3 (Eigendarstellung)

Methodik 2: Die untersuchten ML-Modelle

Tabelle 1: ML-Modell Vergleich

Modell	Vorteil (+)	Nachteil (–)
XGBoost	Höchste Genauigkeit	Anfällig für Overfitting
Deep Gaussian Process	Misst die eigene Unsicherheit	Hoher Rechenaufwand
Symbolic Regression	White-Box: Ergebnis als Formel	Geringste Genauigkeit

Ergebnisse 1: XGBoost mit höchster Vorhersagegenauigkeit

- XGBoost klare Gewinner in Genauigkeit ($R^2 \sim 0.98$).
- DGP ungenauer, bietet aber Unsicherheitsanalyse.
- Symbolic Regression zu hoher Fehler.

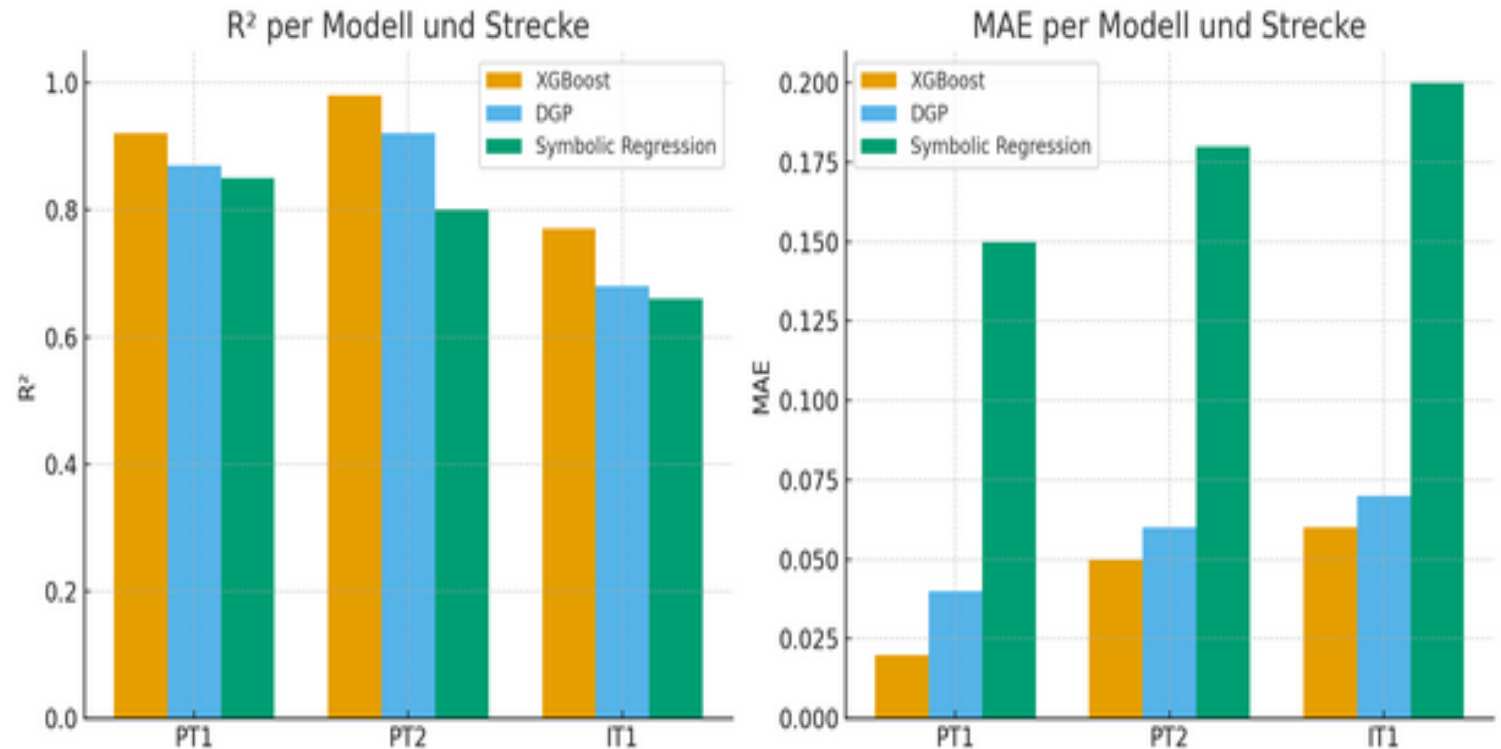


Abbildung 6: ML-Modell Vergleich Plot (Eigendarstellung)

Interaktives Optimierungs-Tool

- ML-Modell ermöglicht multi-kriterielle Optimierung.
- Nutzer kann Präferenzen (z.B. 'schnell' vs. 'Überschwingen') selbst gewichten.

The screenshot displays the 'Interactive Optimization Tool' interface. On the left is a sidebar with a 'Navigation' section containing radio buttons for 'Home', 'Predict PID', 'Evaluate PID', 'Optimize PID' (selected), and 'Simulink Validation'. Below this is a 'Surrogate Model' dropdown set to 'MLP'. The 'Plant Parameter' section includes input fields for 'K (Gain)' (4.50), 'T1' (10.00), 'T2' (3.00), and 'Td (Dead Time)' (0.60), each with minus and plus adjustment buttons. The main area has a blue header 'Use ML-guided optimization to find best PID'. Below it is the 'Define Optimization Weights' section with sliders for 'ISE Weight' (0.50), 'Overshoot Weight' (0.20), 'Settling Time Weight' (0.20), and 'Rise Time Weight' (0.10). The 'Define Performance Constraints' section contains input fields for 'Max ISE' (25.00), 'Max Overshoot (%)' (50.00), 'Max Settling Time' (100.00), 'Max Rise Time' (50.00), and 'Max SSE' (0.50), all with adjustment buttons. A 'Run Optimization' button is at the bottom.

Abbildung 7: Optimierungs-Tool Input (Eigendarstellung)

Interaktives Optimierungs-Tool

Navigation

Choose Mode

- ☐ Home
- ☐ Predict PID
- ☐ Evaluate PID
- ☒ Optimize PID
- ☐ Simulink Validation

Surrogate Model

DGP

Plant Parameter

K (Gain)

4.50

T1

10.00

T2

3.00

Td (Dead Time)

0.60

Time Window [s] (Top 5 Plot)

0.00 100.00

> Debug summary (why fewer than 5?)

Top 5 Distinct PID Controllers (sorted by simulated cost)

	Kp	Ki	Kd	ISE_sim	Overshoot_sim	SettlingTime_sim	RiseTime_sim	SSE_sim	Cost_true
0	2.0357	0.199	5.2862	1.036	1.635	2.903	1.502	0	1.576
1	1.8897	0.1585	4.7383	1.096	0.588	3.704	1.802	0	1.586
2	2.6248	0.2131	6.0532	0.963	5.312	3.804	1.101	0	2.415
3	3.1472	0.2194	6.3748	0.948	11.04	4.204	0.901	0	3.613
4	1.9452	0.1744	3.779	1.215	5.528	9.409	1.902	0	3.785

Step Responses of Top 5 Controllers

Step Response of Top 5 Controllers

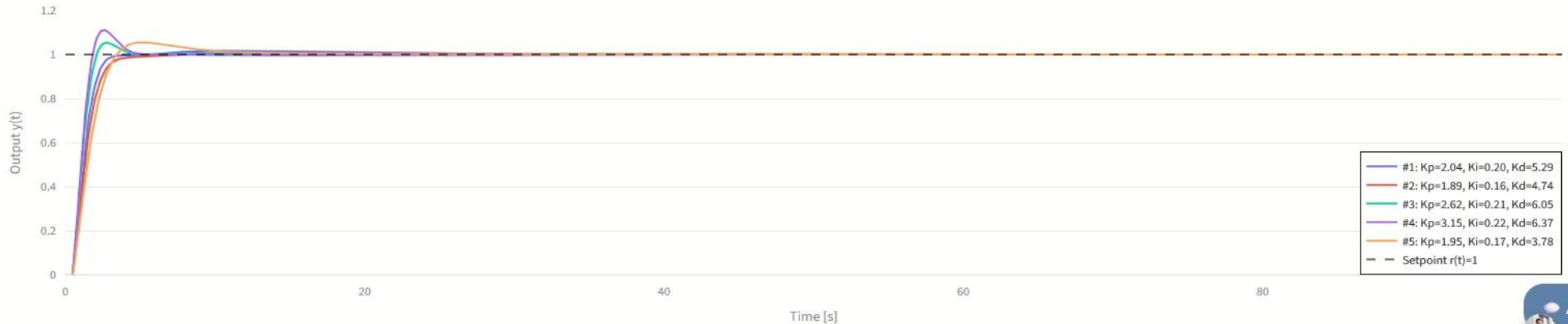


Abbildung 8: Optimierungs-Tool Output (Eigendarstellung)

Ergebnisse 2: Vergleich an synthetischer PT2-Strecke

- ZN (orange): Sehr schnell, aber hohes Überschwingen.
- CHR (lila/grün): Wenig Überschwingen, aber deutlich langsamer.
- ML (blau): Bester Kompromiss aus Geschwindigkeit und Stabilität.

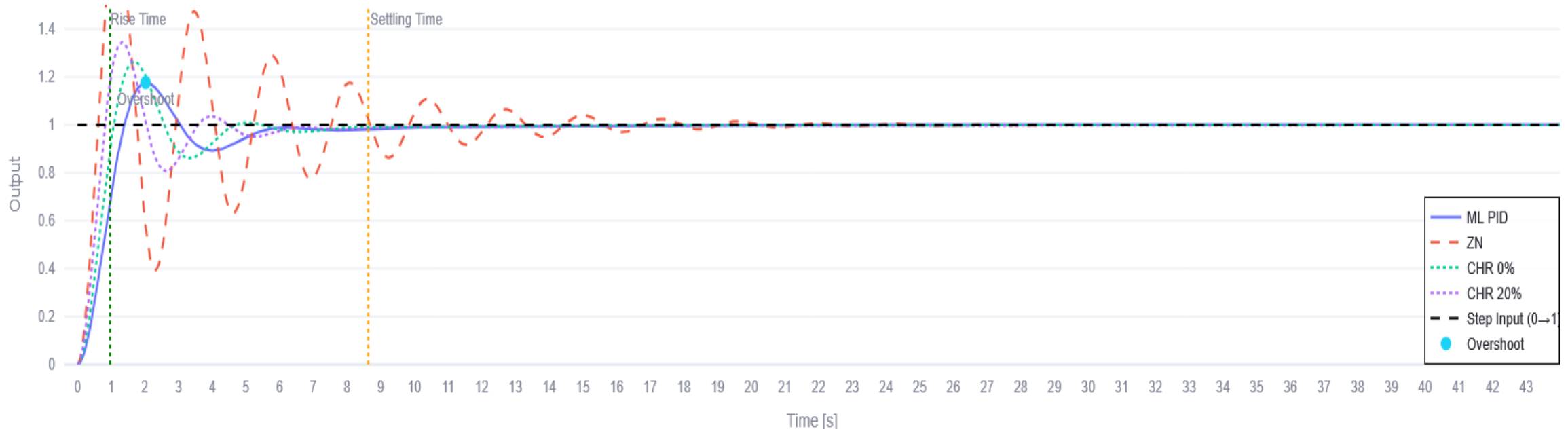


Abbildung 9: Vergleich Synthetische Strecke 1 (Eigendarstellung)

Ergebnisse 3: Test an den Trainingsgrenzen

- ML-Modell wird ungenau oder sogar instabil.
- Wichtige Erkenntnis: Exzellent für Interpolation, aber unzuverlässig bei Extrapolation nach oben!

Closed-Loop Step Response

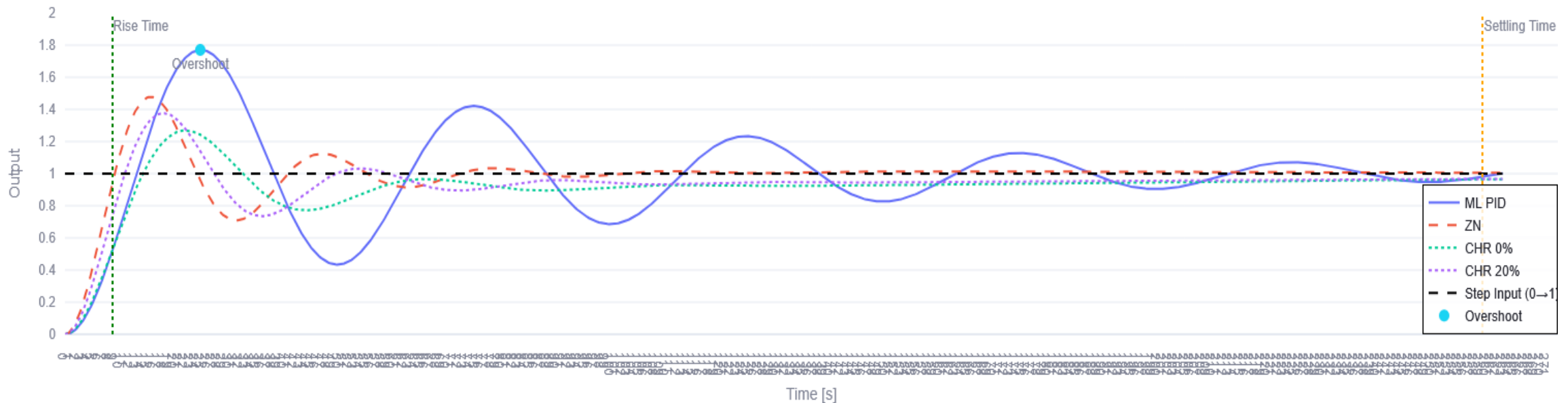


Abbildung 10: Vergleich Synthetische Strecke 2 (Eigendarstellung)

Ergebnisse 4: Validierung an realer PT2 Strecke

- ZN (**rot**): aggressiv, hoher OS, lange, oszillierenden Einschwingzeit
- CHR (**blau/orange**): niedriger OS, mittlere Anstiegszeit, schnelles Einschwingen
- ML-Modell (**grün**): Kaum OS, langsamerer Anstieg, stabilste Regelung.

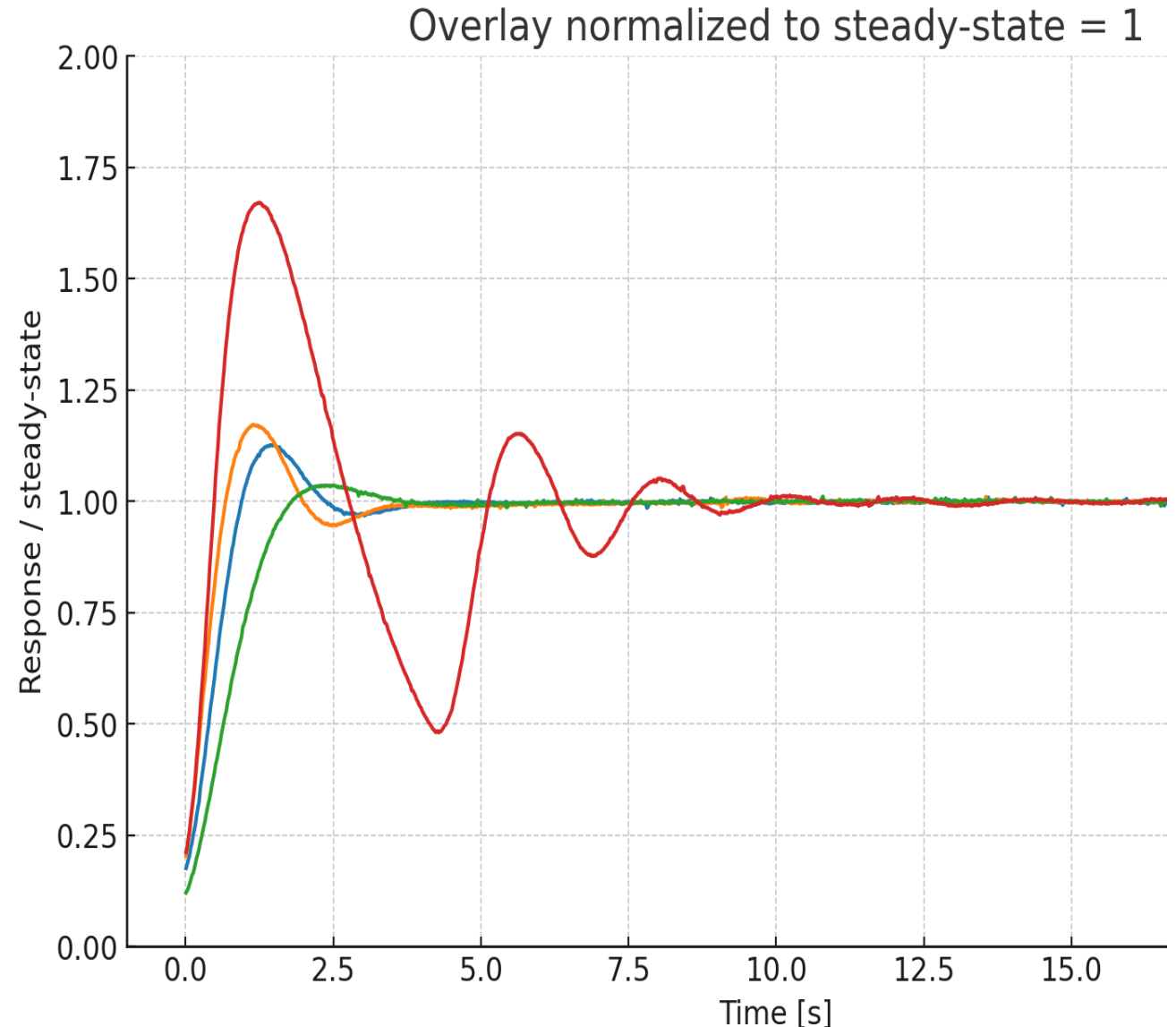


Abbildung 11: Vergleich Reale Strecke 1 (Eigendarstellung)

Ergebnisse 4: Validierung an realer Strecke + Störung

- ML-Regler:
schnellste Erholung
nach Störung Z1+Z2,
sehr hoher initialer
Überschwung.

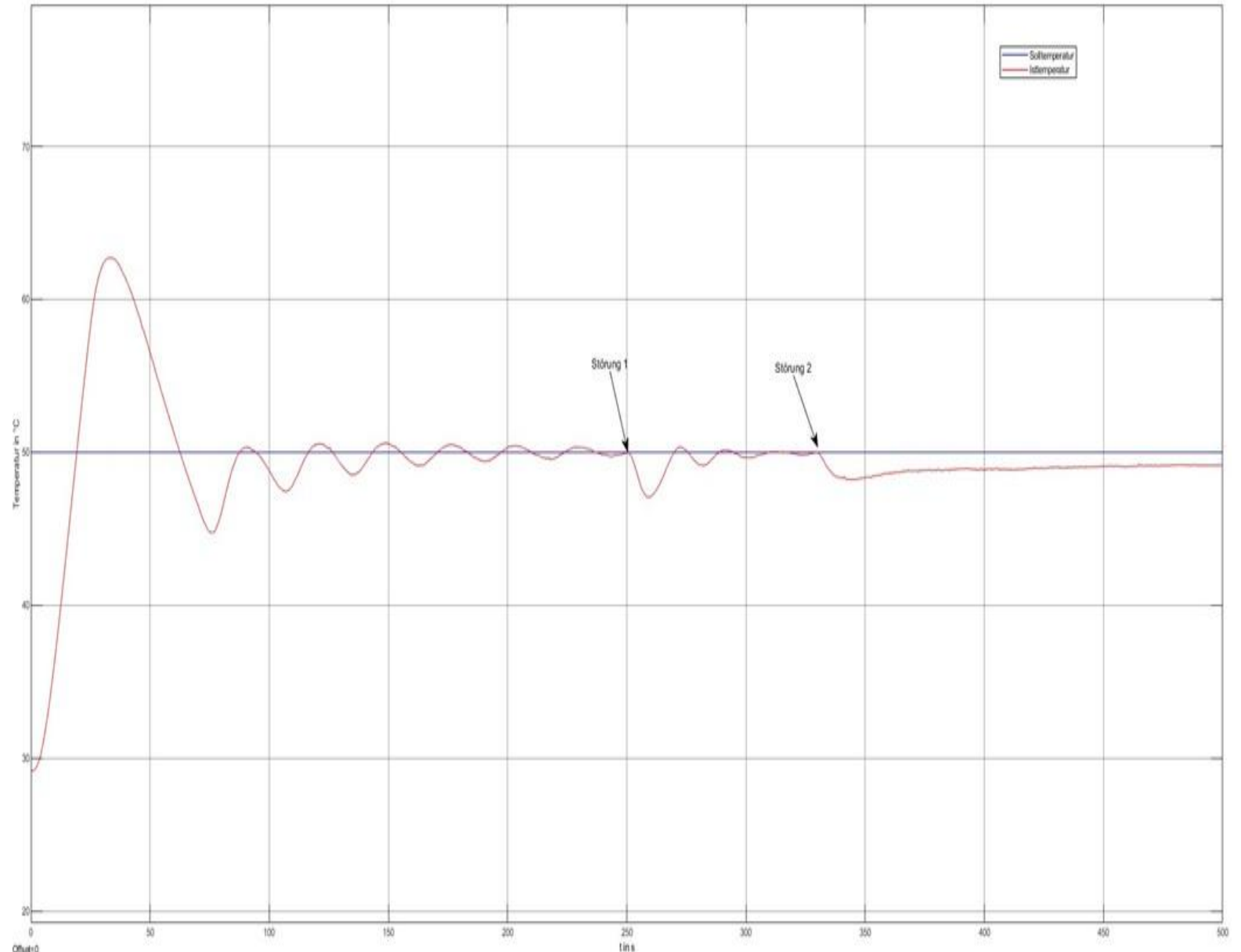


Abbildung 12: Vergleich Reale Strecke 2 ML (Eigendarstellung)

Ergebnisse 4: Validierung an realer Strecke + Störung

- CHR-Regler:
träge und kann sich
nicht stabilisieren,
kein Überschwung,
nicht Oszillierend

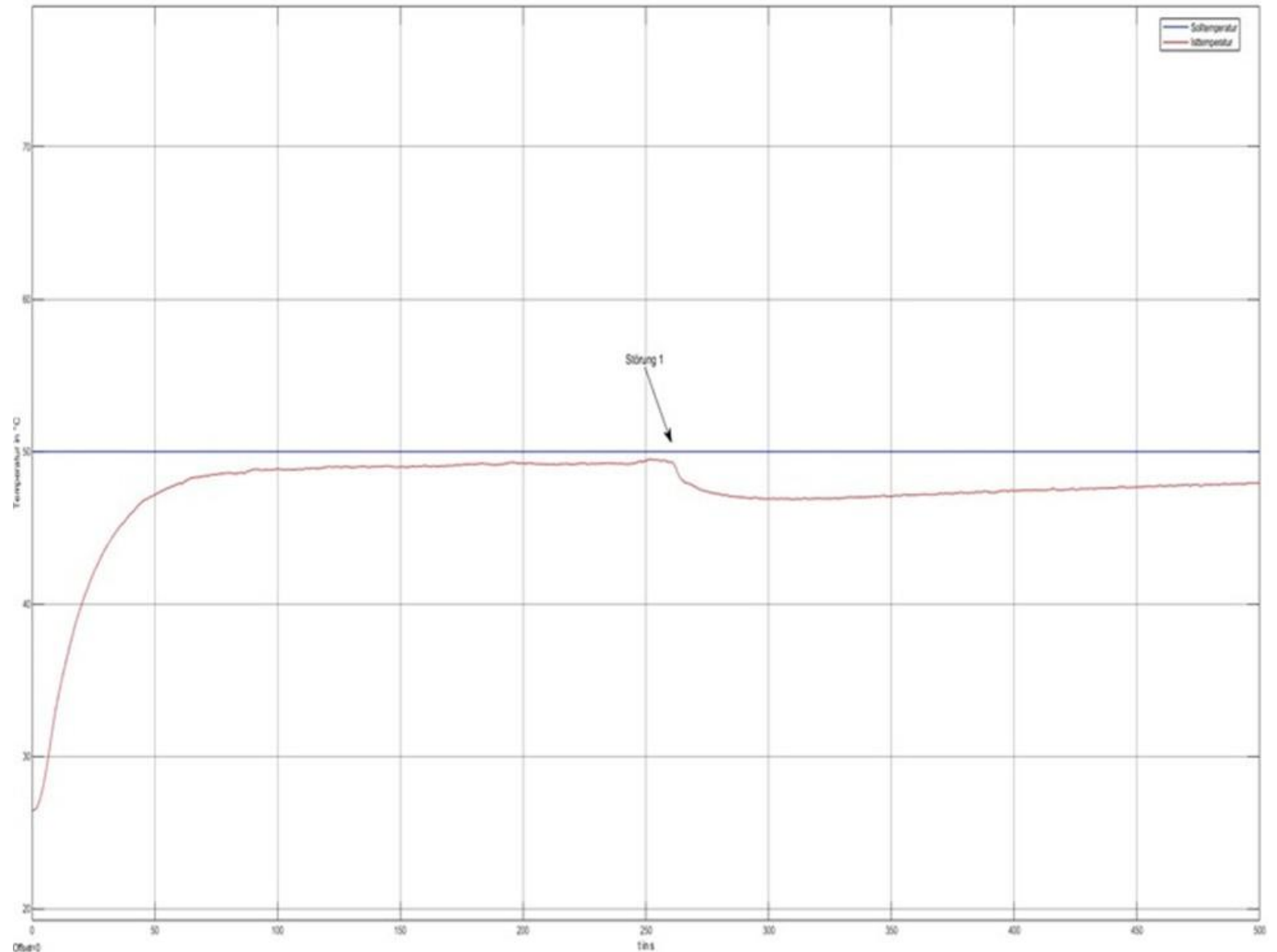


Abbildung 14: Vergleich Reale Strecke 3 CHR (Eigendarstellung)

Diskussion: Stärken & Schwächen

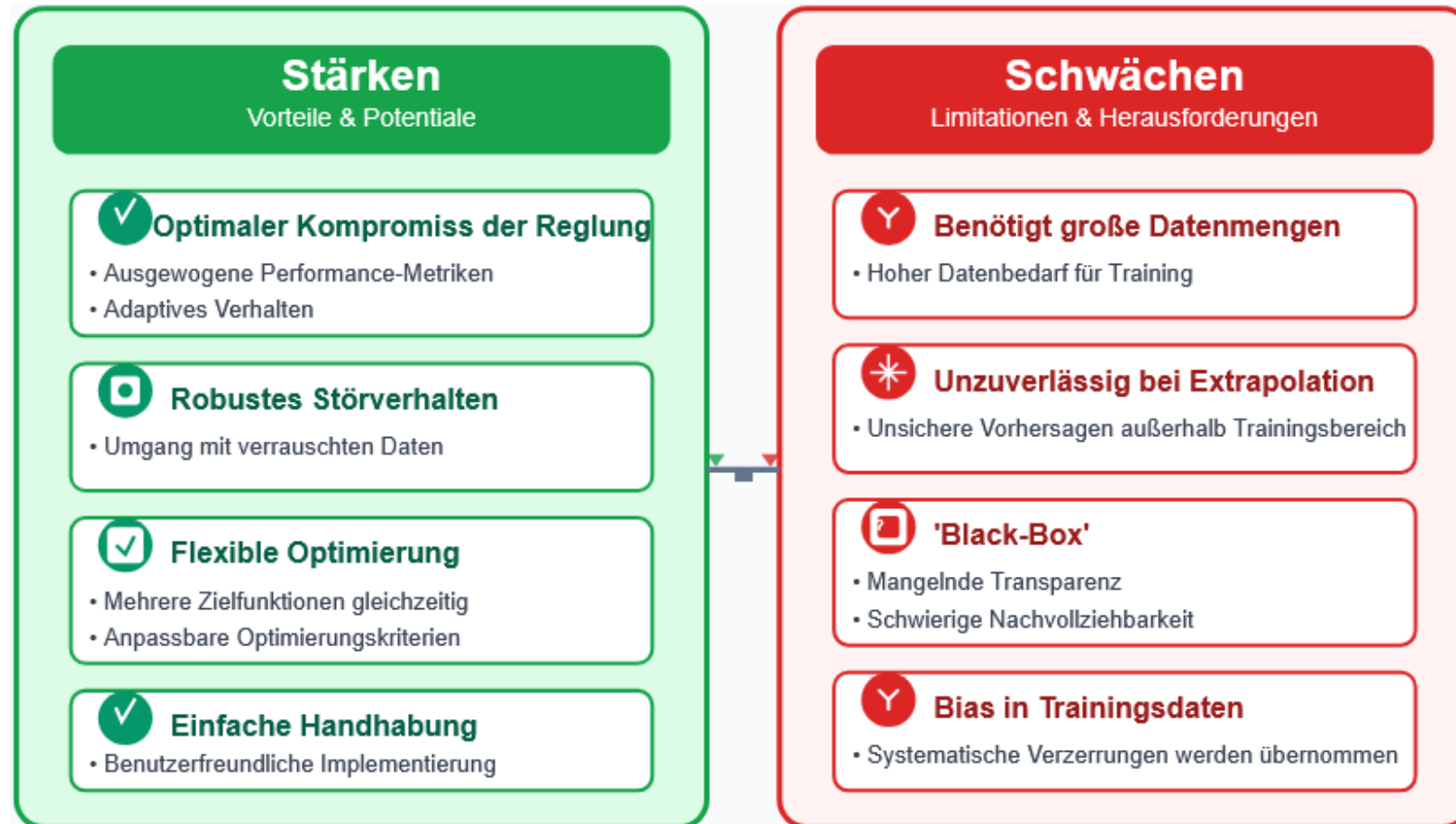


Abbildung 15: Stärken/Schwächen ML (Eigendarstellung)

Fazit & Ausblick

- Fazit:
 - ML-basierte Optimierungsmodelle sind eine leistungsfähige Alternative zu klassischen PID-Einstellverfahren.
- Ausblick:
 - Erweiterung der Trainingsdaten auf komplexere reale Streckentypen.
 - Direktes Lernen aus Messdaten (Ein-/Ausgangssignale).



Vielen Dank für Ihre Aufmerksamkeit!

Fragen & Diskussion

ml-based-pid-optimizer.app